In the next section, we see how we can boost results in action.

2.3.5 Boosting results

We want to give a higher priority (relevance) to certain fields when issuing a query against multiple fields. This way, even if the user doesn't explicitly specify what fields should be boosted, we can provide the best results. Elasticsearch lets us boost the priority of fields in our queries by providing a boost factor next to the fields. For example, to increase the `title` field by a factor of three, we set the boost on the field to `title^3` (the field name followed by caret symbol and a boost factor).

Listing 2.11 A `multi_match` query that boosts a field's importance

```
GET books/_search
{
  "query": {
    "multi_match": {
      "query": "Java",
      "fields": ["title^3","synopsis"]
    }
  }
}
```

Searches through multiple fields

A caret followed by the boost number indicates the field being boosted.

The results will show that the weight of the `title` field's score has increased. This means we bumped the document to a higher position by boosting its score.

We may wish to search for a phrase: for example, "how is the weather in London this morning" or "recipe for potato mash." To do so, we use another type of query called `match_phrase`—coming up next.

2.3.6 Search phrases

At times we wish to search for a sequence of words exactly in a given order, like finding all books that have the phrase "must-have book for every Java programmer" in the `synopsis` field. We can write a `match_phrase` query for this purpose.

Listing 2.12 Searching for books with an exact phrase

```
GET books/_search
{
  "query": {
    "match_phrase": {
      "synopsis": "must-have book for every Java programmer"
    }
  }
}
```

match_phrase query that fetches an exact match of a phrase (sequence of words)

Phrase to search for in each book's synopsis field

This query searches for that sequence of words in the `synopsis` field across all our books and returns the book *Effective Java*:

```
"hits" : [{
  "_score" : 7.300332,
```